

OOP QUICK REVISION NOTES (Ride Booking + Payment System)

USE CASE

Ride Booking Application

Supports:

- Bike Ride
- Car Ride
- Auto Ride

Payments:

- UPI
 - Card
 - Wallet
-

WITHOUT OOP

```
if(rideType.equals("BIKE")) { ... }  
  
if(rideType.equals("CAR")) { ... }  
  
if(paymentType.equals("UPI")) { ... }  
  
if(paymentType.equals("CARD")) { ... }
```

Problems:

- ✗ Huge if-else chains
- ✗ Hard to maintain
- ✗ Hard to extend
- ✗ Tight coupling

✗ Duplicate code

1. ENCAPSULATION

Definition

Hide data and expose behavior.

Purpose

Protect object state.

WITHOUT ENCAPSULATION

```
class Ride {  
    public double fare;  
    public String status;  
}
```

```
ride.fare = -1000;  
ride.status = "RANDOM";
```

Problem:

Anyone can modify internal data.

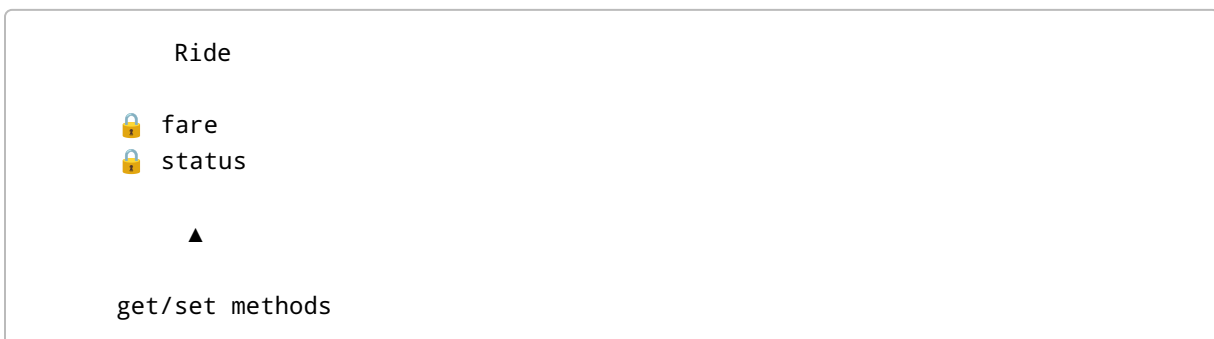
Diagram:

```
Client-1 ---> fare  
Client-2 ---> fare  
Client-3 ---> status
```

WITH ENCAPSULATION

```
class Ride {  
  
    private double fare;  
  
    private String status;  
  
    public void setFare(double fare) {  
  
        if(fare > 0) {  
            this.fare = fare;  
        }  
    }  
  
    public double getFare() {  
        return fare;  
    }  
}
```

Diagram:



Benefits:

- ✅ Data protection
- ✅ Validation
- ✅ Controlled access

One-Line Summary

Encapsulation = Protect Data

2. INHERITANCE

Definition

Acquire common properties from parent.

Purpose

Reuse code.

WITHOUT INHERITANCE

```
class BikeRide {  
  
    startRide();  
  
    endRide();  
}
```

```
class CarRide {  
  
    startRide();  
  
    endRide();  
}
```

```
class AutoRide {  
  
    startRide();  
  
    endRide();  
}
```

Problem:

Duplicate code everywhere.

Diagram:

```
BikeRide
├─ startRide()
└─ endRide()
```

```
CarRide
├─ startRide()
└─ endRide()
```

```
AutoRide
├─ startRide()
└─ endRide()
```

WITH INHERITANCE

```
class Ride {

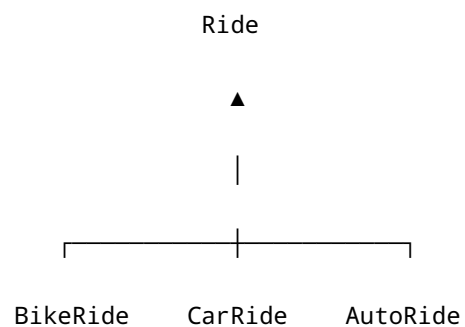
    startRide();

    endRide();

}
```

```
class BikeRide extends Ride {}
class CarRide extends Ride {}
class AutoRide extends Ride {}
```

Diagram:



Benefits:

✓ Reuse code

✓ Avoid duplication

✓ Easier maintenance

One-Line Summary

Inheritance = Reuse Code

3. POLYMORPHISM

Definition

Same method call.

Different behavior.

Purpose

Remove if-else chains.

WITHOUT POLYMORPHISM

```
if(type.equals("BIKE")) {  
    BikeRide ride = new BikeRide();  
    ride.calculateFare();  
}  
else if(type.equals("CAR")) {  
    CarRide ride = new CarRide();  
    ride.calculateFare();  
}
```

Problem:

Every new ride type requires service modification.

Diagram:

```
RideService
```

```
if BIKE  
    BikeRide
```

```
if CAR  
    CarRide
```

```
if AUTO  
    AutoRide
```

WITH POLYMORPHISM

```
abstract class Ride {  
  
    abstract double calculateFare();  
}
```

```
class BikeRide extends Ride {  
  
    double calculateFare() {  
        return 100;  
    }  
}
```

```
class CarRide extends Ride {  
  
    double calculateFare() {  
        return 200;  
    }  
}
```

Service:

```
Ride ride = getRide();  
  
ride.calculateFare();
```

Runtime:

```
Ride ride = new BikeRide();
```

↓

Bike Fare Logic

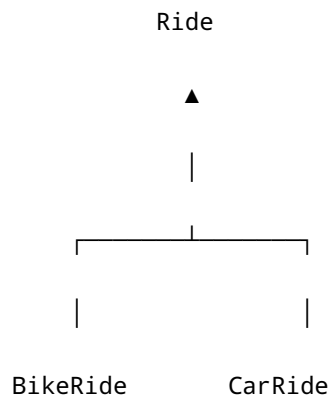
OR

```
Ride ride = new CarRide();
```

↓

Car Fare Logic

Diagram:



```
ride.calculateFare()
```

Benefits:

- ✓ No huge if-else chains
- ✓ Easy extension
- ✓ Cleaner service layer

One-Line Summary

Polymorphism = Same Call + Different Behavior

4. ABSTRACTION

Definition

Hide implementation.

Expose capability.

Purpose

Depend on contract, not implementation.

WITHOUT ABSTRACTION

```
BikeRide bike = new BikeRide();  
  
CarRide car = new CarRide();  
  
AutoRide auto = new AutoRide();
```

Problem:

Service knows every class.

Diagram:

```
RideService  
├─ BikeRide  
├─ CarRide  
├─ AutoRide  
├─ LuxuryRide
```

└ SharedRide

Tightly coupled.

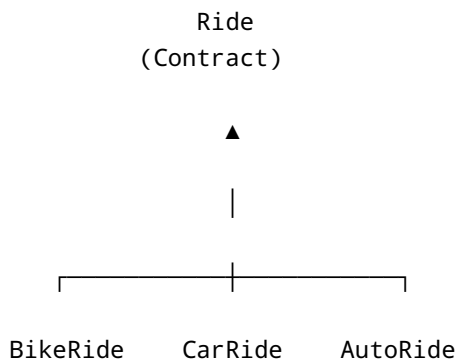
WITH ABSTRACTION

```
abstract class Ride {  
    abstract double calculateFare();  
}
```

Service:

```
Ride ride = getRide();  
ride.calculateFare();
```

Diagram:



Service only knows:

Ride

Benefits:

✓ Loose coupling

✓ Extensible

✓ Easy maintenance

Add:

```
class LuxuryRide extends Ride
```

No service change.

One-Line Summary

Abstraction = Depend On Contract

COMPLETE SYSTEM

Ride Hierarchy

```
abstract class Ride {  
    private double fare;  
  
    public abstract double calculateFare();  
  
    public void startRide() {}  
  
    public void endRide() {}  
}  
  
class BikeRide extends Ride {}  
  
class CarRide extends Ride {}
```

Payment Hierarchy

```
abstract class Payment {  
  
    public abstract void pay(double amount);  
}
```

```
class UpiPayment extends Payment {}  
  
class CardPayment extends Payment {}
```

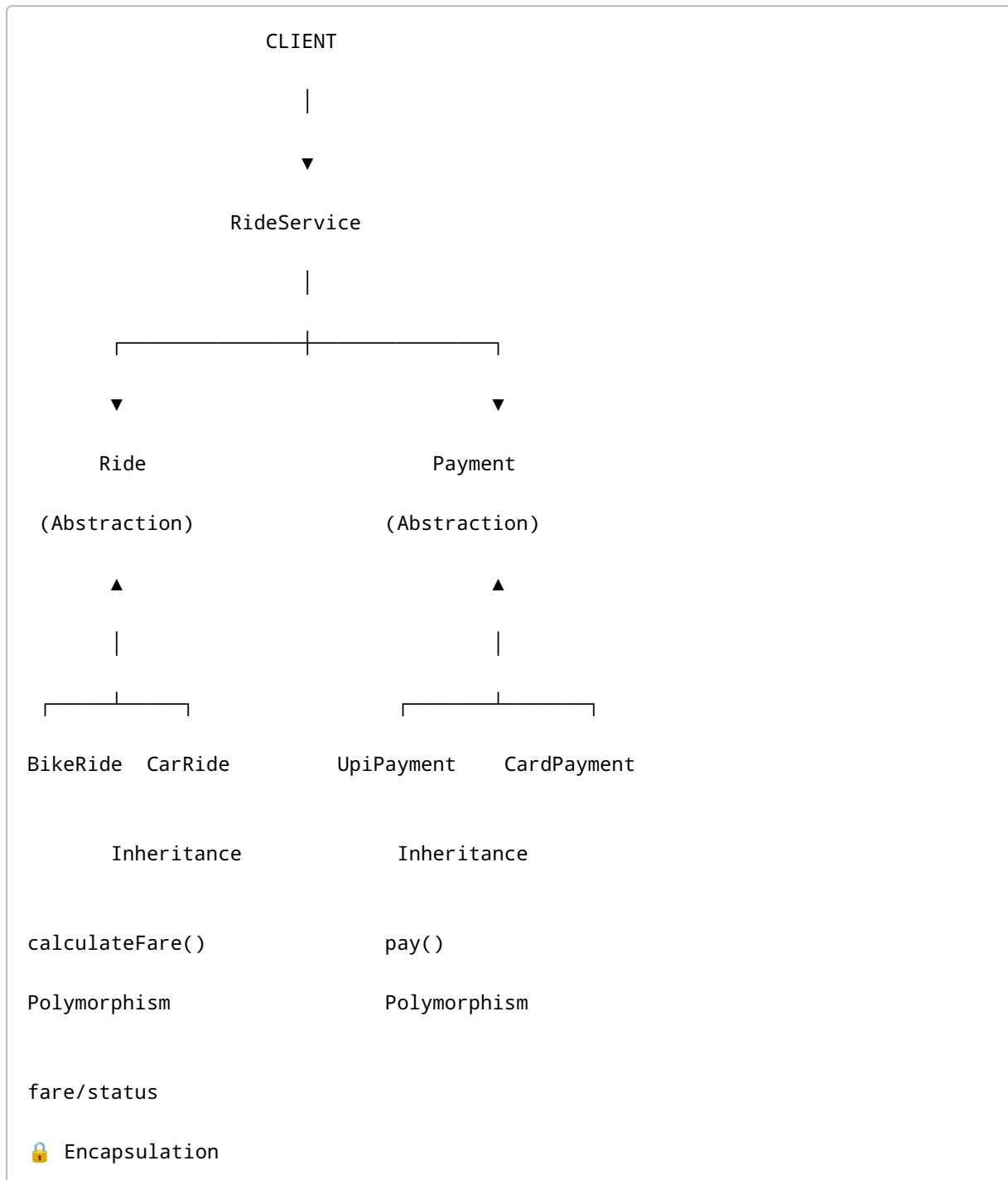
Service Layer

```
class RideService {  
  
    public void completeRide(  
        Ride ride,  
        Payment payment) {  
  
        ride.startRide();  
  
        double fare =  
            ride.calculateFare();  
  
        payment.pay(fare);  
  
        ride.endRide();  
    }  
}
```

Client

```
Ride ride =  
    new BikeRide();  
  
Payment payment =  
    new UpiPayment();  
  
RideService service =  
    new RideService();  
  
service.completeRide(  
    ride,  
    payment  
);
```

FINAL ARCHITECTURE



INTERVIEW REVISION TABLE

Concept	Purpose	Without It
Encapsulation	Protect Data	Anyone can modify object state
Inheritance	Reuse Code	Duplicate code everywhere
Polymorphism	Same Call Different Behavior	Large if-else chains
Abstraction	Depend On Contract	Tight coupling

MEMORY TRICK

Encapsulation = Protect

Inheritance = Reuse

Polymorphism = Replace If-Else

Abstraction = Depend On Contract

ONE-LINE INTERVIEW ANSWER

Encapsulation protects data.

Inheritance reuses code.

Polymorphism removes if-else chains by allowing the same method call to behave differently based on the object.

Abstraction hides implementation details and lets systems depend on contracts instead of concrete classes.